Digital Twin-based Architecture and Design for Co-simulation of Omnidirectional Mobile Robots*

Victor Araújo[1] and Edna Barros[2]

***Abstract*— This paper presents the design and implementation of a digital twin architecture for a four-wheeled omnidirectional robot competing in the RoboCup Small Size League, developed in the MATLAB/Simulink environment using Simscape Multibody. The framework integrates kinematic modeling, multibody dynamics, and realistic contact forces, while importing CAD models through the Simscape Multibody Link to preserve geometric and inertial accuracy. Experimental validation was performed using in-game situation trajectories from a public SSL dataset. Results show that the digital twin achieves trajectory root-mean-square errors on the order of 150–240 mm and maximum positional errors below 350 mm across all scenarios, while maintaining near real-time simulation performance. These findings confirm that the twin can reproduce real robot behavior even under dynamic conditions. This work presents an architecture for modeling omnidirectional robot dynamics and contact interactions within Simscape Multibody, providing a robust foundation for control validation, performance analysis, and parameter optimization.**

I. INTRODUCTION

The concept of the *Digital Twin* (DT) has emerged along with the evolution of the computational power of cyber-physical systems in modern robotics research. This new technology promises tightly coupled virtual and physical systems that enable real-time adaptive control, predictive maintenance, and immersive visualization [4]. Digital Twin is a virtual representation of a physical object, process or system that uses data collected from the real world to simulate, analyze, and control the state of the physical entity. In particular, achieving such an environment in mobile robotics requires seamless integration of middleware connectivity, high-fidelity control models, and interactive 3D environments. Digital twins are a valuable tool for accelerating the development, testing, and deployment of autonomous mobile robots in realistic environments.

Digital twins in robotics aim to improve simulation, control, and predictive maintenance capabilities, yet several challenges remain before their full potential can be realized. According to [6]. Achieving high-fidelity synchronization between physical robots and their virtual counterparts requires ultra-low latency communication and precise data integration. However, the effectiveness of these virtual tools is often limited by a discrepancy between simulated and real-world behavior, which is called a "reality gap" [5].

According to [3], the mismatch between the digital and physical systems prevents the digital twin from accurately reflecting reality. Such discrepancies can arise from two factors: the inability of simulation models to accurately represent reality due to the digital model not having the necessary complexity to capture all the nuances and behaviors of the physical system, or the fact that the existing sensors may not measure some aspects of the physical system. Without convergence and evolution, model-based optimization is debatable, compromising the full potential of Digital Twins [3]. Also, the scalability of digital twin frameworks can be limited by the computational power of real-time multiphysics simulations and the storage of large volumes of historical operational data. Addressing these challenges is essential for transitioning digital twins from experimental testbeds to production-grade robotic systems.

The existing literature on robotics simulation presents several state-of-the-art simulators widely adopted for mobile robotics, each offering distinct advantages and limitations [2]. Gazebo remains among the most popular choices due to its seamless ROS integration and extensive sensor model library. CoppeliaSim, Webots, PyBullet, and MuJoCo are other general-purpose simulators for robots, while specific-purpose simulators such as CARLA focus on autonomous drive research. The NVIDIA IsaacSim is becoming a popular photorealistic simulator, supporting synthetic data generation and reinforcement learning workflows. In contrast, [1] uses MathWorks' Simscape Multibody as the simulation core for their digital twin framework due to its native integration with MATLAB/Simulink, enabling seamless co-simulation with control algorithms, parameter estimation routines, and real-time hardware-in-the-loop (HIL) testing.

Simscape Multibody allows the creation of high-fidelity multibody dynamics models directly from CAD assemblies, preserving kinematic constraints and physical parameters without requiring manual reimplementation in a separate simulation environment. This tight coupling between physical modeling and control development accelerates the iterative design–test–validate cycle, facilitates direct parameter tuning from experimental data, and supports automatic code generation for deployment on embedded platforms. Therefore, for applications where mechanical accuracy, direct integration with control design, and engineering workflow continuity outweigh the need for photorealistic rendering or large-scale multi-agent simulation, Simscape Multibody presents a compelling alternative.

*This work supported by Centro de Informática, Universidade Federal de Pernambuco

[1]Victor Araújo is with Centro de Informática Universidade Federal de Pernambuco, Av. Jorn. Aníbal Fernandes, s/n - Cidade Universitária, Recife - PE, 50740-560. vhssa@cin.ufpe.br

[2]Edna Barros is with Centro de Informática Universidade Federal de Pernambuco, Av. Jorn. Aníbal Fernandes, s/n - Cidade Universitária, Recife - PE, 50740-560. ensb@cin.ufpe.br

This paper presents a comprehensive Digital Twin design architecture for an omnidirectional mobile robot in the context of the RoboCup Small Size League, focusing on high-fidelity dynamic modeling using Simscape Multibody to reduce the gap between simulation and reality. The main contributions of this work are:

- Present an architecture for designing and implementing an omnidirectional mobile robot using Simscape Multibody.
- Detailed methodology for modeling spatial contact forces and friction at the wheel-ground interface, capturing critical slip dynamics in the simulator.
- Quantitative validation of the digital twin's performance through simulation experiments, focusing on the correlation between commanded inputs and the resulting robot's path.

The remainder of this paper is organized as follows. Section II describes the kinematic and dynamic modeling of the omnidirectional mobile robot, including the platform specifications, inverse and forward kinematics, and relevant dynamic considerations. Section III presents the proposed digital twin architecture developed in Simscape Multibody, detailing the system-level integration, multibody model construction, and the modeling of world and contact interactions. Section IV discusses the control system design, outlining the adopted control strategy and its implementation within the Simulink environment. Experimental procedures and validation results are reported in Section V, followed by conclusions and directions for future research in Section VI.

II. OMNIDIRECTIONAL ROBOT KINEMATICS AND DYNAMICS

This section establishes the theoretical foundation of the omnidirectional mobile robot model, presenting the platform used and the kinematics and dynamic rules that govern its motion.

A. Platform Description

The mobile robot platform is a four-wheeled omnidirectional mobile robotic platform used in the context of the RoboCup Small Size League [9]. The chassis is approximately a cylinder of radius 0.18 m (including wheel extensions) and carries all actuation and control components. Each wheel has a diameter of about 50 mm and is an omniwheel with free-spinning rollers oriented at 90° around its circumference, allowing lateral slip. The wheels are mounted at specific angles to allow the robot to carry and kick a golf ball. Each wheel is driven by a Maxon EC-45 Flat 50W BLDC motor with an in-wheel gearbox, providing sufficient torque for carrying the robot's 2.54 kg weight. For sensing, encoders on each motor provide wheel angular position and speed feedback. An inertial measurement unit (IMU) onboard provides orientation and acceleration data to assist in state estimation and closed-loop control. The robot is powered by a LiPo 4S battery and managed by an STM32F767ZI control board. Detailed specifications for the robot are given in the extended team description document [11]



Fig. 1: Four-wheeled omnidirectional mobile robot.

B. Inverse and Forward Kinematics

Kinematics describes the relationship between the wheel's velocities and the robot's resultant velocity, without considering the forces that cause the motion. Because of the holonomic Mecanum wheel design, the robot can move simultaneously in any planar direction and rotate about its vertical axis. We derive the kinematic relationships between the wheel angular velocities ϕ_i (for wheels $i = 1,2,3,4$) and the robot's planar and angular rate according to the Figure 2 schematic diagram.

Consider (v_x, v_y) to be the translational velocity of the robot in its local coordinate frame, and $\dot{\phi}_R$ the angular velocity about the vertical z-axis, which can be found using the right-hand rule. Each wheel $[M_1, M_2, M_3, M_4]$ has radius R and the geometric parameters L denote the distance from the robot's center to each wheel along the x and y directions. The inverse kinematics mapping a desired robot's velocity to individual wheel speeds is obtained by considering the velocity contribution of the platform motion at each wheel [12]. Assume the desired robot velocity vector, $v_{robot} = [v_x, v_y, \dot{\phi}_R]^T$, and the required individual wheel angular velocities, $\dot{\phi}_{wheels} = [\dot{\phi}_1, \dot{\phi}_2, \dot{\phi}_3, \dot{\phi}_4]^T$. For the four-wheeled platform with angles $\theta_1 = 60°$, $\theta_2 = 135°$, $\theta_3 = -135°$, and $\theta_4 = -60°$. This relationship can be expressed as:

$$\begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \\ \dot{\phi}_3 \\ \dot{\phi}_4 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} -\sin(\theta_1) & \cos(\theta_1) & L \\ -\sin(\theta_2) & \cos(\theta_2) & L \\ -\sin(\theta_3) & \cos(\theta_3) & L \\ -\sin(\theta_4) & \cos(\theta_4) & L \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \dot{\phi}_R \end{bmatrix} \tag{1}$$

Forward kinematics performs the inverse transformation, calculating the body velocity from the measured wheel velocities. It is obtained via the pseudo-inverse of the Jacobian matrix from Equation (1). Presenting both transformations demonstrates a complete understanding of the system's kinematics. The complete kinematic description for this robot is detailed in [13].

C. Dynamic Considerations

While a purely kinematic model is sufficient for basic motion planning and control, an accurate dynamic model is needed to capture inertia, friction, and load distribution

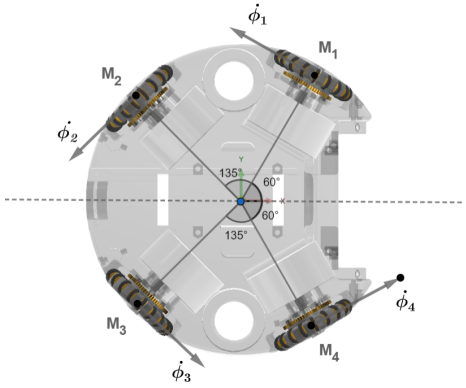


Fig. 2: Schematic diagram of the four-wheeled omnidirectional robot.

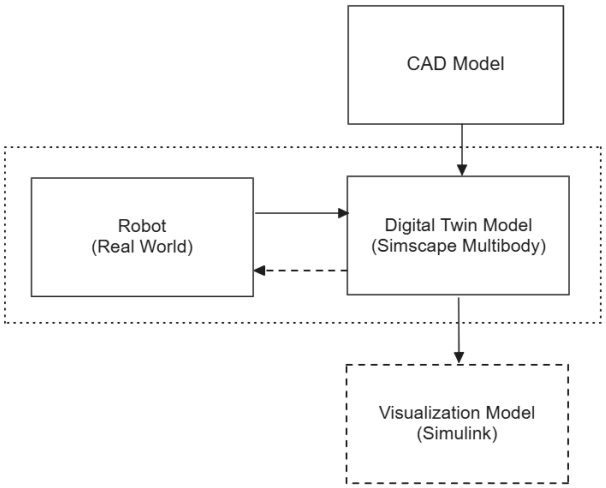


Fig. 3: System architecture block diagram.

effects. The robot's dynamics relate applied forces to the resulting accelerations and motion. In general, the equations of motion for a robotic system can be expressed in joint-space form as

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = B(q)\tau + J^T(q) f_{ext} \qquad (2)$$

The $M(q)$ is the inertia matrix, $C(q,\dot{q})\dot{q}$ represents the Coriolis and centrifugal forces, $g(q)$ is the vector of gravitational forces. On the right-hand side, τ is the vector of actuator inputs (forces) mapped by, $B(q)$ and $J^T(q) f_{ext}$ represents the external forces applied to the system, such as the contact forces with the ground. A complete derivation of this Euler–Lagrange formulation can be found in [7].

For this work, we employ MATLAB's Simscape Multibody toolbox to model and solve the robot's dynamics. Simscape Multibody automatically formulates and numerically solves the multibody equations of motion for the entire mechanical system [10].

III. DIGITAL TWIN ARCHITECTURE IN SIMSCAPE MULTIBODY

This section details the implementation of the digital twin for the omnidirectional mobile robot. We will present the system-level architecture, the architecture of the digital twin, and how we established the connections between the model and the environment.

A. System-Level Architecture

Figure 3 depicts the proposed digital twin architecture. The foundation of the virtual model lies in a CAD representation of the robot, designed in Autodesk Inventor and exported using the Simscape Multibody Link tool[8]. This add-on helps generate the necessary files to describe each component's assembly and geometry, which are then imported into MATLAB/Simulink. This workflow automatically creates a multibody representation with bodies, joints, and constraints that mirror the real-world robot configuration [1]. Once imported, the model is instantiated within Simscape Multibody, where dynamic simulation is performed, and kinematic and inertial properties are parameterized. The resulting digital twin is then connected bidirectionally to the physical robot using wireless communication, allowing sensor data from the real platform to refine the simulation and enabling analyses of robot behavior to optimize robot parameters. Additionally, Simulink provides a built-in visualization tool, ensuring that the CAD-based geometry can be directly inspected and validated against the real robot. By leveraging this integrated workflow, the architecture bridges CAD modeling, physics-based simulation, and visualization in a unified framework, ensuring high-fidelity replication of robot dynamics. The digital twin model also features a simulation mode where data can be imported from the real environment to be used in the Simscape Multibody environment.

B. Multibody Model Implementation

The robot's physical model is implemented in Simscape Multibody with a detailed multibody representation of the robot and its environment. Figure 4 illustrates the main components of the proposed Simscape Multibody architecture. The Field block defines the environment in which the robot operates, modeled according to the official rules and dimensions of the RoboCup Small Size League, thereby providing realistic boundary conditions for testing and validation. The Robot block encapsulates the digital twin developed in Simscape Multibody, fully representing the omnidirectional platform. The Control module implements wheel-level PID controllers, following the design strategy presented by [13], ensuring accurate velocity tracking and stable closed-loop performance. Finally, the Kinematics module performs the inverse kinematics computations, transforming the desired velocities $(v_x, v_y, \dot{\phi}_R)$ into the corresponding wheel angular velocities that serve as reference inputs for the control system. These components form an integrated co-simulation framework that couples environment modeling, robot dynamics, control, and kinematics, enabling high-fidelity digital

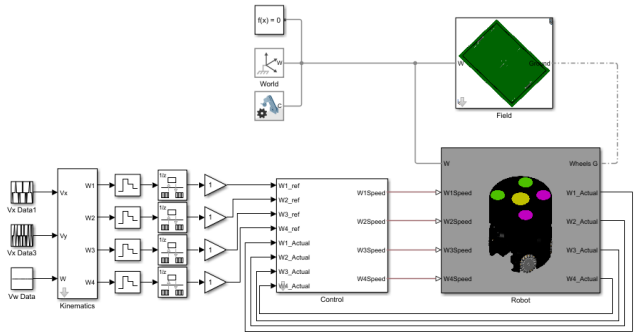


Fig. 4: Co-simulation architecture integrating the Simscape Multibody digital twin with the Simulink control system. In simulation mode, the controller drives the Simscape robot model and receives collected data sensor feedback. In real-time mode, the same controller interfaces with the real robot while the digital twin runs parallel for state estimation and prediction.

twin simulations for RoboCup-oriented experiments.

Using the Simscape Multibody Link tool, we can import the complete CAD assembly of the robot designed in Autodesk Inventor directly into Simulink. This procedure generates an accurate Simscape model in which each link is defined by its precise CAD geometry and mass properties, and all joints and kinematic constraints from the original design are preserved. Following best practices in the literature, such as the approach of [1], we also incorporate key non-ideal dynamics such as joint friction and actuator motor characteristics into the Simscape model to enhance fidelity. Figure 5 details the Robot block from Figure 4, where the chassis was considered a rigid body connected to the four wheels with their respective joints. The Simscape simulation allows modeling of contact interactions or other external forces acting on the robot, so the environment and other obstacles, such as the goal posts, can be represented as part of the physical model.

On the control side, feedback control algorithms run within the Simulink model. The Simulink portion implements the robot's low-level locomotion and parameter estimation controllers. These controllers exchange data with the Simscape plant model at each simulation time step. Specifically, actuator commands such as desired speed are output from Simulink into the Simscape Multibody model through interface blocks. At the same time, sensor feedback is fed back from the Simscape model to the Simulink control logic. This co-simulation arrangement ensures that the control system reads and influences the digital twin as it would interact with a physical robot in closed-loop operation. Because the plant and controller reside in one integrated environment, we can design and tune the control loops with immediate insight into the physical robot's response. In effect, the Simulink-Simscape integration allows rapid prototyping of control strategies on a high-fidelity virtual

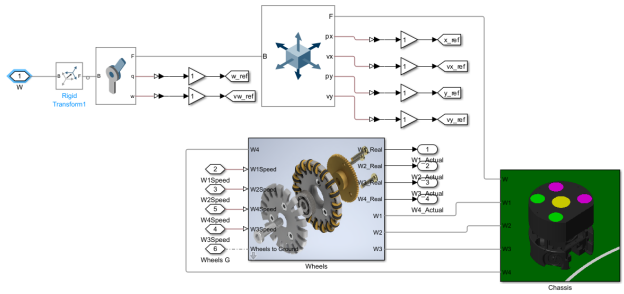


Fig. 5: Multibody representation of the omnidirectional robot in Simscape Multibody. The chassis (center) connects to four wheels via revolute joints. Contact force elements (not visible here) are applied between each wheel and the ground plane. This model was imported from the robot's CAD design, preserving geometry and inertial properties.

robot, which can significantly streamline the development and testing of the robot's software before experimental trials [10].

A key advantage of this architecture is its support for real-time simulation. Simscape employs fixed-step solvers and engine optimizations designed for real-time execution, aligning with the need for real-time simulation in advanced stages of robotic system development. In practice, we can configure the simulation to run at real-time speed, which is crucial for synchronous operation with physical hardware. Therefore, the same digital twin model used in off-line simulation can be executed on a real-time machine, interacting directly with physical components in the loop. Such hardware-in-the-loop (HIL) capability enables a gradual transition from pure simulation to experimentation.

Additionally, Simscape Multibody provides an integrated 3D visualization that animates the imported CAD model during simulation. The robot's motion can be visualized within the simulation, verifying kinematics and dynamics and ensuring the virtual model matches the intended design. However, using the visualization compromises the ability to execute in real-time. However, using the visualization compromises the ability to execute in real-time.

C. World and Contact Modeling

The accuracy of the simulation is critically dependent on the modeling of the interaction between the robot and its environment. The field is represented by a plane block with large dimensions to simulate a flat surface. The physical interaction between each of the four wheels and this ground plane is implemented using the spatial contact force block. This block calculates the regular and friction contact forces based on the collision geometry and material properties.

Within the spatial contact force block, the smooth stick-slip friction model is selected. This model implements an approximation of the Stribeck effect, which is essential for accurately capturing the smooth transition between static and kinetic friction regimes. The transparent parameterization of

the contact model is crucial for scientific reproducibility. Table I details the key parameters used in the simulation.

TABLE I: Key simulation and robot parameters

Parameter	Symbol	Value	Unit
Robot Chassis Mass	M	2.54	kg
Robot Chassis Inertia (z-axis)	I_z	1.25	kg·m^2
Wheel Radius	r	0.0051	m
Distance, Center to Wheel	L	0.08	m
Contact Stiffness	k	1×10^5	N/m
Contact Damping	d	1×10^3	N·s/m
Static Friction Coefficient	μ_s	0.8	-
Kinetic Friction Coefficient	μ_k	0.6	-
Simscape Solver	-	ode23t	-
Maximum Solver Step Size	Δt_{max}	2×10^{-3}	s

IV. Experiments and Results

A. Experimental Scenario

Experiments were conducted using a 6 m $\times$ 4.5 m RoboCup SSL half-field, with markings and boundaries following the official SSL rules [9]. The test setup aligns with the public SSL self-localization dataset [14], which provides ground-truth trajectories collected during in-game situation (igs) runs. For this study, we selected three representative scenarios: igs_01, igs_02, and igs_03, corresponding to maximum commanded velocities of 1.0 m/s, 1.5 m/s, and 2.0 m/s, respectively. Each scenario consists of a continuous trajectory recorded by the league's official vision system, serving as the ground-truth reference for performance evaluation.

B. Performance Evaluation and Visualization

The digital twin was implemented in MATLAB/Simulink using Simscape Multibody (Figure 6), where the same kinematics, dynamics, and control modules described in Section III were preserved. To replicate the dataset experiments, we extracted the desired velocity commands from the dataset logs for igs_01, igs_02, and igs_03. These commands were applied to the Simulink-based controller, which in turn drove the simulated robot model. Simulations were executed with a fixed-step solver at 0.5 kHz to ensure stability and to approximate the timing of the real robot's control loop.

C. Trajectory Tracking Performance

Figures 7a–c show the overlay of ground-truth trajectories (solid red) with the corresponding digital twin trajectories (dashed blue) for igs_01, igs_02, and igs_03. Qualitatively, the simulated paths closely follow the recorded dataset trajectories, though minor deviations become more evident as the commanded velocity increases. In particular, igs_03 at 2.0 m/s exhibits larger lateral deviations, consistent with higher slip effects and unmodeled nonlinearities at higher speeds.

To quantify accuracy, we computed the root-mean-square error (RMSE) and the maximum positional error between ground-truth and digital twin trajectories. Table II summarizes the results. The digital twin achieved sub 200 mm RMSE for scenarios igs_01 and igs_02, while errors increased moderately for igs_03 at higher speeds. Maximum deviations remained below 350 mm in all cases. These results confirm that the Simscape Multibody-based twin could effectively reproduces real robot dynamics under game-like conditions, with discrepancies attributable to unmodeled wheel slip and actuator latency at higher velocities that can be optimized with further algorithms.

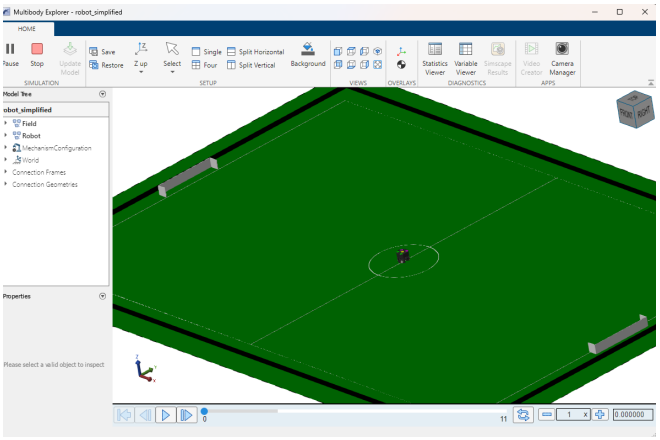


Fig. 6: Digital twin environment implemented in MATLAB/Simulink with Simscape Multibody. The half-field dimensions and goals follow SSL specifications.

D. Discussion

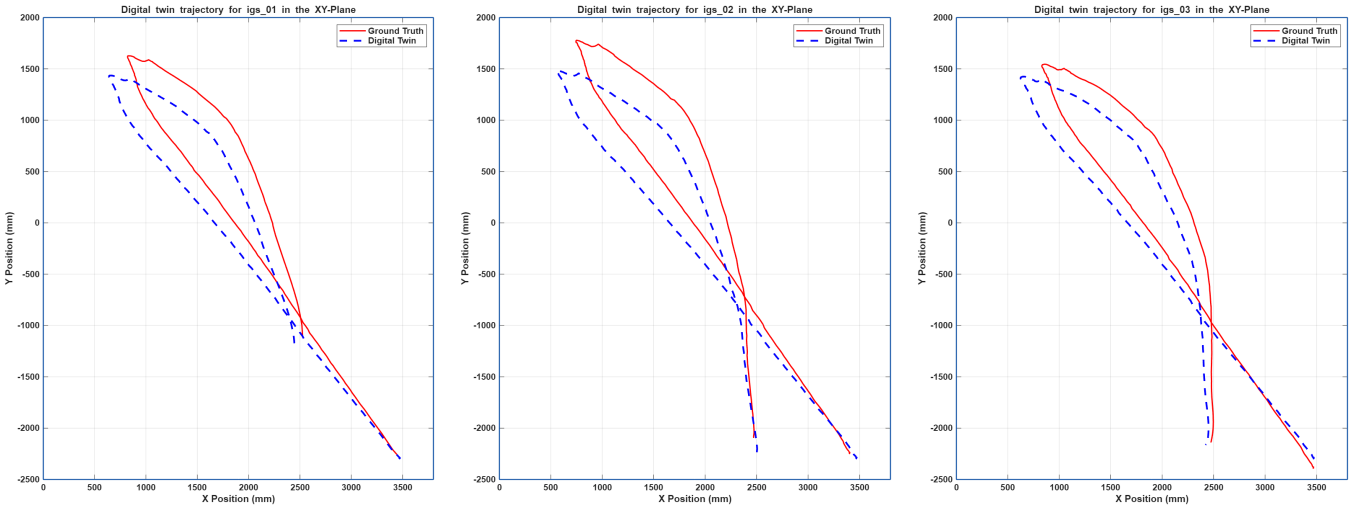
The results demonstrate that the proposed digital twin architecture provides high-fidelity trajectory prediction for SSL robots across multiple in-game scenarios. At low and medium speeds, deviations are low, validating the fidelity of the kinematic and contact models. At higher velocities, errors increase due to slip and dynamic effects not yet fully modeled, suggesting opportunities for incorporating refined friction models and actuator dynamics. Importantly, the simulation maintained near real-time execution, confirming its suitability for future hardware-in-the-loop control validation.

V. Conclusion and Future Work

This paper introduced a MATLAB/Simulink digital twin for a four-wheeled omnidirectional SSL robot that integrates CAD-based multibody modeling, wheel–ground contact, and the robot's kinematics/dynamics. Using SSL in-game scenarios, the experimental evaluation demonstrated that the digital twin is capable of reproducing robot behavior with high accuracy: trajectory RMSE remained on the order of a few centimeters for igs_01 and igs_02, and below 250 mm even in the more challenging high-speed igs_03 scenario. Maximum positional errors remained under 350 mm, confirming that

TABLE II: Quantitative performance metrics for in-game scenarios

Performance Metric	igs_01	igs_02	igs_03
Trajectory RMSE (mm)	149.203	176.510	231.140
Maximum positional error (mm)	241.205	264.341	349.485



(a) igs_01 (1.0 m/s) (b) igs_02 (1.5 m/s) (c) igs_03 (2.0 m/s)

Fig. 7: Comparison between digital twin and experimental trajectories for in-game situations. Ground-truth trajectories (solid red) and digital twin trajectories (dashed blue) are shown for igs_01 (1.0 m/s), igs_02 (1.5 m/s), and igs_03 (2.0 m/s).

the twin captures the essential dynamics of omnidirectional motion while maintaining near real-time performance.

Future work will address the integration of external visualization platforms such as Unity for photorealistic, low-latency visualization. Another research direction is adopting ROS 2 modular communication with physical robots and external tools. Extending the model to include robot subsystems such as the dribbler and kicker will further increase the twin's fidelity and applicability for performance optimization. Finally, scale to multi-robot, match-level simulations to assess robustness and scalability.

REFERENCES

[1] G. Boschetti and T. Sinico, "Designing Digital Twins of Robots Using Simscape Multibody," *Robotics*, vol. 13, no. 4, p. 62, 2024. doi: 10.3390/robotics13040062.

[2] J. Collins, S. Chand, A. Vanderkop, and D. Howard, "A Review of Physics Simulators for Robotic Applications," *IEEE Access*, vol. 9, pp. 51416–51431, 2021. doi: 10.1109/ACCESS.2021.3068769.

[3] C. Cronrath and B. Lennartson, "How Useful is Learning in Mitigating Mismatch Between Digital Twins and Physical Systems?," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 1, pp. 758–770, 2024. doi: 10.1109/TASE.2022.3231386.

[4] M. Grieves and J. Vickers, "Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems," in *Transdisciplinary Perspectives on Complex Systems: New Findings and Approaches*, F.-J. Kahlen, S. Flumerfelt, and A. Alves, Eds. Cham: Springer, 2017, pp. 85–113.

[5] X. Hu, S. Li, T. Huang, B. Tang, R. Huai, and L. Chen, "How simulation helps autonomous driving: A survey of sim2real, digital twins, and parallel intelligence," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 593–612, 2023.

[6] A. Mazumder, M. F. Sahed, Z. Tasneem, P. Das, F. R. Badal, M. F. Ali, M. H. Ahamed, S. H. Abhi, S. K. Sarker, S. K. Das, *et al.*, "Towards next generation digital twin in robotics: Trends, scopes, challenges, and future," *Heliyon*, vol. 9, no. 2, 2023.

[7] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer, 2009.

[8] MathWorks, "Simscape Multibody Link User's Guide," The MathWorks, Inc., Natick, MA, USA, 2023. [Online]. Available: `https://www.mathworks.com/help/physmod/smlink/`

[9] RoboCup Small Size League Committee, "RoboCup Small Size League Rules," 2025. [Online]. Available: `https://robocup-ssl.github.io/ssl-rules/sslrules.pdf` (accessed Aug. 18, 2025).

[10] MathWorks, "Simscape Multibody Documentation," The MathWorks, Inc., Natick, MA, USA, 2025. [Online]. Available: `https://www.mathworks.com/help/sm/index.html` (accessed Aug. 18, 2025).

[11] A. L. de Oliveira, C. A. da Silva Gomes, C. V. S. da Silva, C. M. de Sousa Alves, D. A. M. de Souza, D. P. F. Araujo Xavier, E. P. da Silva, F. B. Martins, L. H. C. Santos, L. D. Maciel, *et al.*, "RoboCIn Small Size League Extended Team Description Paper for RoboCup 2023," *arXiv preprint arXiv:2307.10018*, 2023.

[12] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*. MIT Press, 2011.

[13] V. H. S. S. Araujo, "Uma abordagem para tuning de um controlador PI para motores brushless DC: um estudo de caso aplicado ao controle de movimento de um robô omnidirecional," M.Sc. dissertation, 2020.

[14] J. G. Melo, L. Cavalcanti, R. Joaquim, V. Araújo, and E. Barros, "Dataset and Baseline Experiments for Self-Localization and Tracking in the RoboCup Small Size League," in *2023 Latin American Robotics Symposium (LARS), 2023 Brazilian Symposium on Robotics (SBR), and 2023 Workshop on Robotics in Education (WRE)*, pp. 520–525, IEEE, 2023.